Proyecto #1

Velasco López Aarón Isaías

Debemos realizar un proyecto donde vamos a renderizar uno de los modelos OBJ proporcionados en el classroom (Conejo, Dragón o Buda), se debe utilizar al menos dos luces y una debe estar encendida todo el tiempo y ser de color blanco; la otra luz puede apagarse y encenderse y con un color azul principalmente (debe tener una componente azul con mayor peso). La forma en que se enciende y apaga la segunda luz debe especificarse.

El modelo debe tener dos propiedades de material:

> a) Ambiente {0.00, 0.00, 0.00, 1.00}, Difusa = {0.50, 0.50, 0.50, 1.0} y Especular {0.70, 0.70, 0.70, 1.00} con factor de brillo (exponente) = 32.00 .
> b) Ambiente {0.23125, 0.23125, 0.23125, 1.0}, Difusa = {0.2775, 0.2775, 0.2775, 1.0} y Especular {0.773911, 0.773911, 0.773911, 1.0} con factor de brillo (exponente) = 89.60 .

También cada material debe tener al menos una textura ya sea procedural o en imagen es a elección y para el mapeo de textura se puede mapear sobre todo el modelo o solo parte de él. El método para seleccionar entre un conjunto de propiedades de material y otro se debe especificar. Se debe de implementar un modelo de sombreado Phong y el modelo debe poder observarse desde al menos tres puntos de vista y/o posiciones distintas de la cámara. La selección de las diferentes cámaras se debe realizar con teclas.

Para esto usamos lo siguiente:

Archivos OBJ: Hacemos la lectura del OBJ para cargar el modelo desde un archivo externo, está compuesto de vértices, las normales y las caras del objeto, donde los vértices indican los puntos en el espacio 3D, las normales indican dónde apunta la superficie en cada parte del modelo y las caras cómo se conectan los puntos para formar la geometría del OBJ.

Vértices, normales y caras: El OBJ se representa mediante vértices usando coordenadas:

- x posición horizontal
- y posición vertical
- z profundidad

También se usan normales para la iluminación y caras triangulares para formar la superficie completa del modelo, los triángulos conectan tres vértices y, junto con sus normales, permite calcular correctamente la iluminación.

Normalización del modelo: Para los modelos grandes las coordenadas pueden ser muy grandes, muy pequeñas o desplazadas, calculamos su caja envolvente para obtener su

centro y su tamaño máximo. Lo trasladamos al origen y lo escalamos automáticamente para quedar centrado y visible dentro de la escena sin depender del tamaño original.

Vectores y operaciones matemáticas: Los vectores en 3D representan posiciones, direcciones, normales, luces y cámaras, también los ocupamos para las operaciones (suma, resta, producto punto, producto cruz, cálculo de longitud y normalización), son la base del renderizado ya que calculan la orientación, ángulos, iluminación y proyecciones.

Coordenadas homogéneas: Usamos los vectores de cuatro componentes (x, y, z, w) en lugar de trabajar solo con tres dimensiones para aplicar las matrices de transformación del pipeline 3D y realizar la división final por w.

Matrices 4x4: Se usa en la construcción del pipeline geométrico, con las matrices se realizan traslaciones, escalamiento, rotaciones, cambio de cámara, proyección en perspectiva y proyección ortogonal y las coordenadas de pantalla, esto permite que todas las transformaciones se hagan dentro del renderizador.

Transformaciones geométricas: Se utilizan para modificar la orientación y posición del OBJ dentro del espacio 3D, aplica las rotaciones sobre los ejes “x” y “y” para observar el modelo desde diferentes ángulos y para centrar y escalar el OBJ antes de dibujarlo.

Pipeline de renderizado 3D: Transforma el modelo OBJ con la matriz, después pasa a la matriz de vista para colocarlo respecto a la cámara, luego se aplica la matriz de proyección y finalmente la matriz de viewport para convertirlo a coordenadas de pantalla para que al final de la proyección se divide entre w.

Sistema de cámara: Se usa para representar el punto donde se observa la escena, cada cámara tiene una posición, punto al que mira y un vector que define cuál es la orientación vertical del sistema.

Proyección 3D a 2D: Convierte las coordenadas 3D en coordenadas 2D para poder dibujarlas en la ventana, se crearon dos proyeccione:

- Proyección ortogonal: no tiene efecto de perspectiva y los objetos no se ven más pequeños por estar lejos
- Proyección en perspectiva: los objetos lejanos se ven más pequeños y es más parecido a la vista de las personas

Viewport: Se usa para llevar los puntos a las coordenadas reales de pantalla, las cuales son las posiciones de los píxeles dentro del framebuffer
.
Rasterización: Es el proceso donde los triángulos del OBJ se convierten en píxeles en la pantalla para rellenar toda la superficie del modelo y no solo dibujar sus bordes, se hace triángulo por triángulo, recorriendo los píxeles de la región donde se encuentra cada triángulo.

Coordenadas baricéntricas: Las coordenadas baricéntricas se usan para comprobar si un píxel pertenece a un triángulo y para interpolar datos dentro de él para calcular

correctamente la profundidad, la normal, la posición del punto y los datos de textura para cada píxel.

Interpolación corregida por perspectiva: Se utilizó la interpolación corregida por perspectiva para evitar deformaciones visuales cuando el OBJ se observa con proyección perspectiva, esto es importante porque los atributos interpolados no deben interpolarse de forma lineal simple en pantalla cuando existe perspectiva.

framebuffer: Se utiliza una imagen en memoria para guardar el resultado del render antes de mostrarlo en la ventana.

Z-buffer: Guarda la profundidad de cada píxel para determinar qué parte del OBJ está delante y cuál está detrás.
.
Back-face culling: Se utiliza un descarte de caras traseras para evitar rasterizar triángulos que están orientados en sentido opuesto a la cámara para mejorar el rendimiento y evita dibujar partes del modelo que no deberían verse desde la posición actual.

Vectores normales: Se utilizan para saber hacia dónde apunta cada parte de la superficie del OBJ, calculan la iluminación y permiten medir el ángulo entre la superficie y las luces.

Producto punto: Se usa para medir qué tan alineados están dos vectores.

Modelo de iluminación: Se usa para calcular cómo afecta la luz a cada parte del OBJ, el color final de cada píxel se obtiene combinando varios componentes de iluminación.

Componente ambiental: Es la luz general del ambiente que ilumina ligeramente toda la superficie sin recibir luz directa.

Iluminación difusa: Representa la luz que llega a una superficie dependiendo del ángulo entre la dirección de la luz y la normal, si la superficie apunta más hacia la luz, se ilumina más y si apunta menos, se ilumina menos.

Iluminación especular: Representa el brillo y los reflejos que aparecen cuando la luz rebota hacia la cámara, se necesita:

- la dirección de la luz
- la orientación de la superficie
- la posición del observador
-

Sombreado Phong-Blinn: Se usa para la iluminación del modelo, esta se calcula directamente en cada píxel del triángulo y no solo por cara o por vértice, lo que produce transiciones más suaves y un resultado visual más detallado.

Material: Los materiales modifican la forma en que la luz interactúa con la superficie del OBJ.

Fuentes de iluminación:Las fuentes de iluminación sirven para observar cómo cambia la iluminación del OBJ al exponerse a distintas fuentes de luz.

Textura procedural: Son texturas que no provienen de una imagen externa sino que se generan mediante fórmulas matemáticas.

Textura de imagen: El color base de cada punto del OBJ se obtiene leyendo un píxel de una imagen cargada en memoria. Esto permite darle al OBJ una apariencia visualmente más llamativa sin depender exclusivamente de patrones generados por código.

Mapeo esférico: El mapeado esférico se usa para calcular automáticamente las coordenadas u,v de textura, para esto toma la dirección del punto sobre el OBJ y la convierte en coordenadas bidimensionales para poder leer correctamente la imagen de textura sin modificar el archivo original.

Imagen de textura: Se usa una imagen como textura y se carga desde archivo y se guarda en memoria, durante el renderizado, cada píxel visible del modelo usa sus coordenadas de textura para buscar el color correspondiente dentro de la imagen.

Resolución interna: Se usa para que el renderizado pudiera hacerse al 100%, 75% o 50% del tamaño del widget para mejorar el rendimiento en modelos grandes, ya que un framebuffer más pequeño reduce el número de píxeles que deben procesarse.

Renderizador por software: El renderizado se realiza en CPU y en memoria.

Qt Widgets: Se usa Qt para crear la ventana, los controles de interfaz, el temporizador y para mostrar la imagen final generada en memoria.

Interacción mediante teclado y controles gráficos: Se utiliza el teclado para una interacción rápida y una interfaz gráfica con sliders, listas desplegables y casillas de selección.

Temporizador: Se usa un temporizador para la rotación automática del OBJ, actualizando continuamente la escena cuando esta opción está activada.

Carga y recarga de archivos: El programa puede volver a cargar el modelo OBJ y la imagen de textura durante la ejecución.

Pintado en pantalla: Una vez que el framebuffer termina de generarse, se dibuja dentro del widget usando QPainter, mostrando en pantalla el resultado final del renderizador junto con información del estado actual del programa.

Estructura del código

Primero incluimos las librerías para el funcionamiento, usamos las librerías de Qt QApplication, QMainWindow, QWidget, QPainter, QImage, QSlider, QComboBox, QCheckBox, QPushButton, QTimer, QFile y QTextStream, para poder crear la ventana principal, los controles de la interfaz, el temporizador para la rotación automática, leer el archivo OBJ y dibujar en pantalla la imagen final generada por el renderizador. También utilizamos las librerías estándar de C++ vector, cmath, algorithm y limits, las cuales sirven para almacenar la geometría del modelo, realizar operaciones matemáticas del pipeline 3D, controlar valores mínimos y máximos y manejar el z-buffer.

Ahora definimos las estructuras de datos principales que organizan la información del programa, los vectores tridimensionales para representar posiciones, direcciones y normales en el espacio, los vectores de cuatro componentes que se usan para el pipeline homogéneo con matrices 4x4, la estructura de color para trabajar con componentes RGB en punto flotante, la estructura de matriz 4x4 para las transformaciones geométricas, la estructura de cara para almacenar los índices de los triángulos del modelo OBJ, la estructura de material que guarda los coeficientes ambiental, difuso, especular y el brillo, la estructura de cámara para definir desde dónde se observa la escena y la estructura de luz para representar las fuentes de iluminación blanca y azul.

Declaramos las variables principales del renderizador donde se almacenan los vértices, normales y caras cargadas desde el archivo OBJ, las cámaras disponibles, los materiales, las dos luces, la textura de imagen y también las variables de control que cambian la proyección, la cámara activa, la resolución interna, la rotación del OBJ, el uso del alambre, la activación de la textura y el encendido o apagado de la luz azul. En esta parte calculamos y guardamos los datos como el centro del OBJ y el factor de escala que normaliza automáticamente el OBJ para que quede posicionado dentro de la escena.

Luego implementamos las funciones matemáticas que usamos para el pipeline de renderizado, donde aparecen operaciones como producto punto, producto cruz, cálculo de longitud, normalización de vectores, limitación de valores entre 0 y 1 y control de colores, estas funciones se usan tanto en la parte geométrica como en la iluminación. Después de eso se ponemos las funciones relacionadas con matrices como la matriz identidad, la multiplicación de matriz por matriz, la multiplicación de matriz por vector de cuatro componentes, así como las matrices de traslación, escalamiento y rotación en los ejes “x”, “y” y “z”, de igual forma construimos la matriz de la cámara, la matriz de proyección en perspectiva, la matriz de proyección ortogonal y la matriz de viewport para el pipeline homogéneo completo.

Después hacemos la función encargada de cargar el archivo OBJ, en esta parte se abre el archivo, lo lee línea por línea y separa la información correspondiente a vértices, normales y caras, al encontrar una línea de vértice, guarda sus coordenadas en la lista de vértices. Cuando encuentra una línea de normal, la almacena en la lista de normales del modelo y cuando encuentra una cara, extrae los índices de sus esquinas y la convierte en triángulos si fuera necesario. Después de cargar la geometría, el programa calcula normales de respaldo en caso de que hicieran falta, detecta si el archivo realmente trae normales

utilizables y se hace la caja envolvente del OBJ para calcular el centro y la escala con los que se normaliza y con todo esto se puede colocar correctamente en pantalla sin depender de su tamaño original. Posteriormente se implementa la carga de la imagen de textura y esta función intenta abrir el archivo de imagen, convertirlo y guardarlo en memoria para que después pueda leerse píxel por píxel durante el renderizado, si la imagen no se encuentra, el programa puede seguir funcionando sin una textura activa.

Ahora creamos las funciones que modifican el comportamiento del programa desde la interfaz gráfica, generamos los métodos que cambian la rotación en “x”y en “y”, el tipo de proyección, el uso del alambre, la activación de la rotación automática, el material activo, la cámara seleccionada, la luz azul, la resolución interna y el uso de la textura. Cada vez que uno de estos valores cambia, el widget vuelve a dibujarse para mostrar el nuevo resultado de la escena en tiempo real. Continuando generamos el núcleo del renderizador, que se encuentra en el método encargado de pintar la escena, donde primero se verifica que el OBJ sí haya sido cargado y luego se crea un framebuffer usando un QImage, así como el z-buffer que guardará la profundidad más cercana en cada píxel.

Seleccionamos la cámara y el material activos, y construimos las matrices principales del pipeline (la matriz de normalización del modelo, la matriz de rotación que depende de los controles del usuario, la matriz de vista de la cámara, la matriz de proyección y la matriz de viewport), las matrices se multiplican para obtener la transformación completa que lleva los puntos del objeto hasta coordenadas de pantalla, incluyendo la división por w, luego se define la función que proyecta cada vértice del OBJ, esta función convierte el punto del OBJ en un vector homogéneo de cuatro componentes, le aplica la matriz de modelo, la matriz de vista y la matriz de proyección, obtiene el valor de w, realiza la división perspectiva y finalmente usa la matriz de viewport para obtener sus coordenadas en pantalla.

Además de la posición 2D que se genera debemos de preparar los otros atributos que serán necesarios durante la rasterización, como la posición del punto en el OBJ, la posición en el espacio mundo y la normal transformada, todos ellos almacenados de forma adecuada para permitir interpolación corregida por perspectiva, ahora se implementa la función que obtiene el color base del modelo mediante textura de imagen y como el archivo OBJ no tiene coordenadas UV, el programa calcula automáticamente las coordenadas u y v usando un mapeo esférico y para lograr esto toma la posición del punto normalizado sobre el OBJ, obtiene un ángulo horizontal con atan2 y una coordenada vertical con asin, y con esos valores localiza el píxel correspondiente dentro de la imagen de textura. De esta forma el OBJ puede usar una textura de imagen.

A continuación creamos el modelo de iluminación calculando el color final de cada píxel combinando la componente ambiental, la difusa y la especular, usando la normal del píxel, la dirección hacia cada luz y la dirección hacia la cámara, donde se aplica el modelo Phong-Blinn, ya que la iluminación se calcula por píxel y no solamente por cara o por vértice. Para esto primero se parte del color base obtenido por la textura, luego se suma el aporte de la luz blanca fija y después, si está activada, el aporte de la luz azul dando el resultado final se limita al rango válido y después se convierte a formato entero para ser almacenado en el buffer de color.

Para generar la rasterización de triángulos se recorren todas las caras del OBJ y para cada una se obtienen sus tres vértices y sus tres normales, calculamos la normal de la cara y se aplica descarte de caras traseras para evitar rasterizar triángulos que no miran hacia la cámara todo esto para proyectar los tres vértices a pantalla y se calcular el rectángulo mínimo que encierra al triángulo. Dentro de ese rectángulo se recorren todos los píxeles y se utilizan coordenadas baricéntricas para comprobar si el píxel actual pertenece o no al triángulo, Si éste pertenece se interpola la profundidad y se compara con el z-buffer pero si el nuevo fragmento está más cerca de la cámara, se reconstruyen la posición, la normal y las coordenadas necesarias con corrección por perspectiva que obtiene un color base de la textura, se calcula la iluminación y finalmente se escribe el color resultante en el framebuffer. Si el modo alámbrico está activado, también se dibujan las aristas del triángulo para visualizar la malla.

Generamos las funciones auxiliares relacionadas con el archivo OBJ y las normales que procesan los índices de vértices y normales que aparecen en las caras del OBJ, se interpretan correctamente índices positivos o negativos, y se define una rutina de respaldo para calcular normales suaves por vértice cuando el OBJ no las proporciona o cuando alguna esquina no tuviera referencia válida, incluimos una función que decide qué normal usar en cada esquina del triángulo, dando prioridad a la normal original del OBJ y recurriendo a la normal calculada únicamente si es necesario, se construye la interfaz principal del programa mediante la clase de la ventana que crea el área de renderizado y el panel lateral con todos los controles los cuales son los sliders para la rotación, combos para seleccionar proyección, material, cámara, resolución interna y repetición de textura, casillas para activar alambre, rotación automática, luz azul y textura, y un botón para recargar el modelo y la imagen.

También se conectan todos esos controles con los métodos del widget de render para que cualquier cambio se refleje inmediatamente en pantalla, en esta misma parte se configura un temporizador que llama periódicamente a la función de rotación automática para animar el modelo cuando así lo indique el usuario y por último se define la función principal main, que es el punto de entrada del programa donde se crea la aplicación Qt, se instancia la ventana principal, se muestra en pantalla y se inicia el ciclo de eventos donde a partir de ese momento la aplicación queda en ejecución, respondiendo tanto a las acciones del usuario como a las actualizaciones necesarias para volver a dibujar la escena.

Código Ejecutable

```
#include <QApplication>
#include <QMainWindow>
#include <QWidget>
#include <QPainter>
#include <QImage>
#include <QSlider>
#include <QLabel>
#include <QComboBox>
#include <QCheckBox>
#include <QPushButton>
#include <QHBoxLayout>
#include <QVBoxLayout>
#include <QTimer>
#include <QMessageBox>
#include <QFile>
#include <QTextStream>
#include <QKeyEvent>
#include <QDir>
#include <QCoreApplication>
#include <QStringList>
#include <vector>
#include <cmath>
#include <algorithm>
#include <limits>

// ========================
// VECTORES, COLORES Y MATRICES
// ========================

// Generamos un vector de 3 componentes para las posiciones,
direcciones y normales
struct Vec3 {
    float x, y, z;
    Vec3() : x(0), y(0), z(0) {}
    Vec3(float X, float Y, float Z) : x(X), y(Y), z(Z) {}

    // Se suman dos vectores.
    Vec3 operator+(const Vec3& o) const { return Vec3(x + o.x, y + o.y,
z + o.z); }

    // Se restan dos vectores.
```

```
    Vec3 operator-(const Vec3& o) const { return Vec3(x - o.x, y - o.y,
z - o.z); }

    // Se multiplica el vector por un escalar.
    Vec3 operator*(float s) const { return Vec3(x * s, y * s, z * s); }

    // Dividimos el vector entre un escalar.
    Vec3 operator/(float s) const { return Vec3(x / s, y / s, z / s); }
};

// Creamos un vector de 4 componentes para el pipeline matricial
struct Vec4 {
    float x, y, z, w;
    Vec4() : x(0), y(0), z(0), w(0) {}
    Vec4(float X, float Y, float Z, float W) : x(X), y(Y), z(Z), w(W)
{}
};

// Generamos los colores RGB en punto flotante
struct Color3 {
    float r, g, b;
    Color3() : r(0), g(0), b(0) {}
    Color3(float R, float G, float B) : r(R), g(G), b(B) {}

    // Se suman dos colores
    Color3 operator+(const Color3& o) const { return Color3(r + o.r, g
+ o.g, b + o.b); }

    // se multiplica un color por un escalar
    Color3 operator*(float s) const { return Color3(r * s, g * s, b *
s); }
};

// Generamos la matriz 4x4 para lo que seria el modelo, vista,
proyección y viewport
struct Mat4 {
    float m[4][4];
    Mat4() {
        for (int r = 0; r < 4; ++r)
            for (int c = 0; c < 4; ++c)
                m[r][c] = 0.0f;
    }
};
```

```
// ========================
// UTILIDADES MATEMÁTICAS BÁSICAS
// ========================

// Producto punto
static float dot(const Vec3& a, const Vec3& b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

// Producto cruz
static Vec3 cross(const Vec3& a, const Vec3& b) {
    return Vec3(
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    );
}

// Longitud de un vector
static float lengthVec(const Vec3& v) {
    return std::sqrt(dot(v, v));
}

// Devolvemos el vector normalizado
static Vec3 normalize(const Vec3& v) {
    float len = lengthVec(v);
    if (len < 1e-8f) return Vec3(0, 0, 0);
    return v * (1.0f / len);
}

// Limitamos los rangos a 0 y 1
static float clamp01(float x) {
    if (x < 0.0f) return 0.0f;
    if (x > 1.0f) return 1.0f;
    return x;
}

// Limitamos tambien el color a un rango visible
static Color3 clampColor(const Color3& c) {
    return Color3(clamp01(c.r), clamp01(c.g), clamp01(c.b));
}
```

```
// Se devuelve el mayor de tres valores
static float max3(float a, float b, float c) {
    return std::max(a, std::max(b, c));
}

// ========================
// MATRICES 4x4 DEL PIPELINE
// ========================

// Generamos la matriz identidad
static Mat4 identity() {
    Mat4 r;
    for (int i = 0; i < 4; ++i) r.m[i][i] = 1.0f;
    return r;
}

// Multplicamos dos matrices 4x4
static Mat4 multiplyMat4(const Mat4& a, const Mat4& b) {
    Mat4 r;
    for (int i = 0; i < 4; ++i) {
        for (int j = 0; j < 4; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < 4; ++k) sum += a.m[i][k] * b.m[k][j];
            r.m[i][j] = sum;
        }
    }
    return r;
}

// Multiplicamos una matriz 4x4 por un vector 4D
static Vec4 multiplyVec4(const Mat4& m, const Vec4& v) {
    return Vec4(
        m.m[0][0] * v.x + m.m[0][1] * v.y + m.m[0][2] * v.z + m.m[0][3]
* v.w,
        m.m[1][0] * v.x + m.m[1][1] * v.y + m.m[1][2] * v.z + m.m[1][3]
* v.w,
        m.m[2][0] * v.x + m.m[2][1] * v.y + m.m[2][2] * v.z + m.m[2][3]
* v.w,
        m.m[3][0] * v.x + m.m[3][1] * v.y + m.m[3][2] * v.z + m.m[3][3]
* v.w
    );
}
```

```
// Matriz de traslacion
static Mat4 translation(float tx, float ty, float tz) {
    Mat4 r = identity();
    r.m[0][3] = tx;
    r.m[1][3] = ty;
    r.m[2][3] = tz;
    return r;
}

// Matriz de escalamiento
static Mat4 scaling(float sx, float sy, float sz) {
    Mat4 r = identity();
    r.m[0][0] = sx;
    r.m[1][1] = sy;
    r.m[2][2] = sz;
    return r;
}

// Matriz de rotacion en x
static Mat4 rotationX(float a) {
    Mat4 r = identity();
    float c = std::cos(a), s = std::sin(a);
    r.m[1][1] = c;  r.m[1][2] = -s;
    r.m[2][1] = s;  r.m[2][2] = c;
    return r;
}

// Matriz de rotacion en y
static Mat4 rotationY(float a) {
    Mat4 r = identity();
    float c = std::cos(a), s = std::sin(a);
    r.m[0][0] = c;  r.m[0][2] = s;
    r.m[2][0] = -s; r.m[2][2] = c;
    return r;
}

// Matriz de rotacion en z
static Mat4 rotationZ(float a) {
    Mat4 r = identity();
    float c = std::cos(a), s = std::sin(a);
    r.m[0][0] = c;  r.m[0][1] = -s;
    r.m[1][0] = s;  r.m[1][1] = c;
    return r;
```

```
}

// Hacemos una camara con un estilo lookAt
static Mat4 lookAt(const Vec3& eye, const Vec3& center, const Vec3& up)
{
    Vec3 f = normalize(center - eye);
    Vec3 s = normalize(cross(f, up));
    Vec3 u = cross(s, f);

    Mat4 r = identity();
    r.m[0][0] = s.x;  r.m[0][1] = s.y;  r.m[0][2] = s.z;  r.m[0][3] =
-dot(s, eye);
    r.m[1][0] = u.x;  r.m[1][1] = u.y;  r.m[1][2] = u.z;  r.m[1][3] =
-dot(u, eye);
    r.m[2][0] = -f.x; r.m[2][1] = -f.y; r.m[2][2] = -f.z; r.m[2][3] =
dot(f, eye);
    return r;
}

// Generamoms la matriz de proyeccion en perspectiva
static Mat4 perspective(float fovyRadians, float aspect, float zNear,
float zFar) {
    Mat4 r;
    float t = std::tan(fovyRadians * 0.5f);
    float f = 1.0f / std::max(t, 1e-6f);
    r.m[0][0] = f / aspect;
    r.m[1][1] = f;
    r.m[2][2] = (zFar + zNear) / (zNear - zFar);
    r.m[2][3] = (2.0f * zFar * zNear) / (zNear - zFar);
    r.m[3][2] = -1.0f;
    return r;
}

// De igual forma lo hacemos con una matriz para la proyeccion
ortogonal
static Mat4 orthographic(float left, float right, float bottom, float
top, float zNear, float zFar) {
    Mat4 r = identity();
    r.m[0][0] = 2.0f / (right - left);
    r.m[1][1] = 2.0f / (top - bottom);
    r.m[2][2] = -2.0f / (zFar - zNear);
    r.m[0][3] = -(right + left) / (right - left);
    r.m[1][3] = -(top + bottom) / (top - bottom);
```

```
    r.m[2][3] = -(zFar + zNear) / (zFar - zNear);
    return r;
}

// Hacemos la matriz de viewport para pasar de NDC a pixeles
static Mat4 viewport(float width, float height) {
    Mat4 r = identity();
    r.m[0][0] = width * 0.5f;
    r.m[0][3] = width * 0.5f;
    r.m[1][1] = -height * 0.5f;
    r.m[1][3] = height * 0.5f;
    r.m[2][2] = 0.5f;
    r.m[2][3] = 0.5f;
    return r;
}

// Extraemos lo que son x, y, z de un vector 4D
static Vec3 xyz(const Vec4& v) {
    return Vec3(v.x, v.y, v.z);
}

// ========================
// ESTRUCTURAS DEL MODELO Y LA ESCENA
// ========================

// Vemos una esquina de la cara del OBJ
struct FaceVertex {
    int v = -1;
    int vn = -1;
};

// Por cada cara que es triangular guardamos tres esquinas
struct Face {
    FaceVertex a, b, c;
};

// Decidimos los materiales para el modelo phong
struct Material {
    Color3 ka;
    Color3 kd;
    Color3 ks;
    float shininess;
};
```

```
// Damos los datos de la camara
struct CameraData {
    Vec3 eye;
    QString name;
};

// Damos los datos de la luz
struct LightData {
    bool enabled;
    Vec3 pos;
    Color3 intensity;
};

// ==========================
//      WIDGET DE RENDER
// ==========================

class RenderWidget : public QWidget {
public:
    explicit RenderWidget(QWidget* parent = nullptr) : QWidget(parent)
{
        // Tamaño mínimo y foco para captar teclas.
        setMinimumSize(920, 680);
        setFocusPolicy(Qt::StrongFocus);

        // LAs camara ya con un modelo normalizado
        cameras.push_back({Vec3(0.0f, 0.15f, 4.20f), "Camara 1"});
        cameras.push_back({Vec3(2.60f, 1.20f, 4.60f), "Camara 2"});
        cameras.push_back({Vec3(-3.00f, 1.70f, 3.80f), "Camara 3"});

        // Material A
        materials.push_back({
            Color3(0.00f, 0.00f, 0.00f),
            Color3(0.50f, 0.50f, 0.50f),
            Color3(0.70f, 0.70f, 0.70f),
            32.0f
        });

        // Material B
        materials.push_back({
            Color3(0.23125f, 0.23125f, 0.23125f),
            Color3(0.2775f, 0.2775f, 0.2775f),
```

```
            Color3(0.773911f, 0.773911f, 0.773911f),
            89.60f
        });

        // Luz blanca
        whiteLight = {true, Vec3(2.8f, 3.2f, 4.6f), Color3(1.0f, 1.0f,
1.0f)};

        // Luz azul
        blueLight  = {true, Vec3(-3.0f, 1.8f, 2.2f), Color3(0.20f,
0.35f, 0.90f)};
    }

    // CArgamos el OBJ, leemos los vertices, normales, caras, y por
ultimo preparamos el modelo.
    bool loadOBJ(const QString& filename) {
        QFile file(filename);
        if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            return false;
        }

        // Por si se necesita limpiar algun modelo anterior
        vertices.clear();
        objNormals.clear();
        computedNormals.clear();
        faces.clear();
        hasOBJNormals = false;

        // Reinicia los ,imites del modelo
        minBound = Vec3( 1e9f,  1e9f,  1e9f);
        maxBound = Vec3(-1e9f, -1e9f, -1e9f);

        QTextStream in(&file);
        while (!in.atEnd()) {
            QString line = in.readLine().trimmed();
            if (line.isEmpty() || line.startsWith('#')) continue;

            QStringList parts = line.split(' ', Qt::SkipEmptyParts);
            if (parts.isEmpty()) continue;

            // Genera la linea del vertice
            if (parts[0] == "v" && parts.size() >= 4) {
```

```
                Vec3 p(parts[1].toFloat(), parts[2].toFloat(),
parts[3].toFloat());
                vertices.push_back(p);

                minBound.x = std::min(minBound.x, p.x);
                minBound.y = std::min(minBound.y, p.y);
                minBound.z = std::min(minBound.z, p.z);
                maxBound.x = std::max(maxBound.x, p.x);
                maxBound.y = std::max(maxBound.y, p.y);
                maxBound.z = std::max(maxBound.z, p.z);
            }
            // Línea de normal del archivo OBJ
            else if (parts[0] == "vn" && parts.size() >= 4) {
                Vec3 n(parts[1].toFloat(), parts[2].toFloat(),
parts[3].toFloat());
                objNormals.push_back(normalize(n));
            }
            // Línea de cara
            else if (parts[0] == "f" && parts.size() >= 4) {
                std::vector<FaceVertex> idx;
                idx.reserve(parts.size() - 1);
                for (int i = 1; i < parts.size(); ++i) {
                    idx.push_back(parseFaceVertexToken(parts[i]));
                }

                // Cuando llega un triangulo se guarda directo
                if (idx.size() == 3) {
                    faces.push_back({idx[0], idx[1], idx[2]});
                }
                //Cuando llega un quad se triangula
                else if (idx.size() == 4) {
                    faces.push_back({idx[0], idx[1], idx[2]});
                    faces.push_back({idx[0], idx[2], idx[3]});
                }
                // Cuando llega un polígono mayor lo triangulamos en un
abanico
                else if (idx.size() > 4) {
                    for (size_t i = 1; i + 1 < idx.size(); ++i) {
                        faces.push_back({idx[0], idx[i], idx[i + 1]});
                    }
                }
            }
        }
```

```
        // Calculamos las normales suaves por si el OBJ si no tiene
normales utilizables
        computeFallbackNormals();

        // Detecta si al menos algunas esquinas sí traen normal del
archivo.
        hasOBJNormals = !objNormals.empty();
        for (const Face& f : faces) {
            if (f.a.vn >= 0 || f.b.vn >= 0 || f.c.vn >= 0) {
                hasOBJNormals = true;
                break;
            }
        }

        // Calculamos el centro y escala para que se normalice el
modelo.
        modelCenter = (minBound + maxBound) * 0.5f;
        Vec3 size = maxBound - minBound;
        float maxDim = max3(std::fabs(size.x), std::fabs(size.y),
std::fabs(size.z));
        normalizeScale = (maxDim > 1e-8f) ? (2.0f / maxDim) : 1.0f;

        update();
        return !vertices.empty() && !faces.empty();
    }

    // Carga una imagen como textura
    bool loadTextureImage(const QString& filename) {
        QImage img;
        if (!img.load(filename)) {
            textureLoaded = false;
            textureImage = QImage();
            update();
            return false;
        }

        // Convertimos a RGB32 para que la lectura de los pixeles sea
mas simple.
        textureImage = img.convertToFormat(QImage::Format_RGB32);
        textureLoaded = true;
        update();
        return true;
```

```
    }

    // Cambia la rotacion en x desde el slider
    void setAngleX(int value) { angleX = value * 3.14159265f / 180.0f;
update(); }

    // Cambia la rotacion el y desde el slider
    void setAngleY(int value) { angleY = value * 3.14159265f / 180.0f;
update(); }

    // Cambia entre perspectiva y ortogonal
    void setPerspective(bool value) { usePerspective = value; update();
}

    // Activa o desactiva el alambre
    void setWireframe(bool value) { wireframe = value; update(); }

    // Activa o desactiva la rotacion automatica
    void setAutoRotate(bool value) { autoRotate = value; }

    // Cambia el material
    void setMaterialIndex(int idx) { materialIndex = idx; update(); }

    // Cambia la camara
    void setCameraIndex(int idx) { cameraIndex = idx; update(); }

    // Enciende o apaga la luz azul
    void setBlueLightEnabled(bool value) { blueLight.enabled = value;
update(); }

    // Quita y agrega la textura de imagen
    void setTextureEnabled(bool value) { textureEnabled = value;
update(); }

    // Cambia cuantas veces podemos poner la textura en el modelo
    void setTextureRepeatIndex(int idx) {
        if (idx == 0) textureRepeat = 1.0f;
        else if (idx == 1) textureRepeat = 2.0f;
        else textureRepeat = 3.0f;
        update();
    }
```

```
    // Puede cambiar la resolucion del modelo para que sea mas facil de
renderizar
    void setRenderScaleIndex(int idx) {
        if (idx == 0) renderScale = 1.0f;
        else if (idx == 1) renderScale = 0.75f;
        else renderScale = 0.50f;
        update();
    }

    // Avanza la rotacion automatica si está activa
    void stepRotation() {
        if (autoRotate) {
            angleY += 0.015f;
            update();
        }
    }

protected:
    // teclas para elegir entre camaras, luz azul y vista de alambre.
    void keyPressEvent(QKeyEvent* event) override {
        if (event->key() == Qt::Key_1) {
            cameraIndex = 0;
            update();
        } else if (event->key() == Qt::Key_2) {
            cameraIndex = 1;
            update();
        } else if (event->key() == Qt::Key_3) {
            cameraIndex = 2;
            update();
        } else if (event->key() == Qt::Key_B) {
            blueLight.enabled = !blueLight.enabled;
            update();
        } else if (event->key() == Qt::Key_W) {
            wireframe = !wireframe;
            update();
        } else if (event->key() == Qt::Key_T) {
            textureEnabled = !textureEnabled;
            update();
        } else {
            QWidget::keyPressEvent(event);
        }
    }
```

```
    // Genera todo el render que se vera en el software
    void paintEvent(QPaintEvent*) override {
        if (vertices.empty() || faces.empty()) {
            QPainter p(this);
            p.fillRect(rect(), QColor(18, 18, 26));
            p.setPen(Qt::white);
            p.drawText(rect(), Qt::AlignCenter, "No se encontro el
OBJ");
            return;
        }

        // Podemos bajar la resolucion del render para mejorar la
velocidad
        const int renderW = std::max(320, int(width() * renderScale));
        const int renderH = std::max(240, int(height() * renderScale));

        // Framebuffer interno
        QImage img(renderW, renderH, QImage::Format_RGB32);
        img.fill(QColor(18, 18, 26));

        // Z-buffer para saber qué pixel está más cerca
        std::vector<float> zbuf(renderW * renderH,
std::numeric_limits<float>::infinity());

        // Elige el material y la camara que el usuario queira
        CameraData cam = cameras[cameraIndex];
        Material mat = materials[materialIndex];

        // Generamos una funcion auxiliar para las barycentricas y
prueba de interior del triangulo
        auto edge = [](float ax, float ay, float bx, float by, float
px, float py) {
            return (px - ax) * (by - ay) - (py - ay) * (bx - ax);
        };

        // ========================
        // MATRICES DEL PIPELINE 4x4
        // ========================

        // Normalizamos el modelo, primero lo centralizamos y lo
escalamos
        Mat4 normalizeTranslate = translation(-modelCenter.x,
-modelCenter.y, -modelCenter.z);
```

```
        Mat4 normalizeScaleMat = scaling(normalizeScale,
normalizeScale, normalizeScale);
        Mat4 normalizeModel = multiplyMat4(normalizeScaleMat,
normalizeTranslate);

        // Transformaciones del usuario para la rotacion
        Mat4 rotX = rotationX(angleX);
        Mat4 rotY = rotationY(angleY);
        Mat4 rotZ = rotationZ(0.0f);

        // Como se genera el modelo final
        Mat4 model = multiplyMat4(rotY, multiplyMat4(rotX,
multiplyMat4(rotZ, normalizeModel)));

        // Vista de la camara tipo lookAt
        Vec3 center(0.0f, 0.0f, 0.0f);
        Vec3 up(0.0f, 1.0f, 0.0f);
        Mat4 view = lookAt(cam.eye, center, up);

        // Proyección elegida por el usuario
        float aspect = (renderH > 0) ? float(renderW) / float(renderH)
: 1.0f;
        Mat4 projection = usePerspective
            ? perspective(55.0f * 3.14159265f / 180.0f, aspect, 0.10f,
50.0f)
            : orthographic(-1.8f * aspect, 1.8f * aspect, -1.8f, 1.8f,
0.10f, 50.0f);

        // Viewport para pasar de NDC a pixeles de la imagen
        Mat4 viewportMat = viewport(float(renderW), float(renderH));

        // Combinaciones utiles de matrices
        Mat4 viewModel = multiplyMat4(view, model);
        Mat4 mvp = multiplyMat4(projection, viewModel);

        // Estructura con datos preparados para rasterizacion
        struct ScreenV {
            float x, y;
            float depth;
            float invW;
            Vec3 objOverW;
            Vec3 worldOverW;
            Vec3 normalOverW;
```

```
        };

        // Tomamos una imagen 2D y la aplicamos con un mapeo esférico
        auto sampleBaseColor = [&](const Vec3& PobjNorm, const Vec3&
Pworld) -> Color3 {
            // Si el usuario quita la textura o la imagen no existe,
pone un degradado neutro
            if (!textureEnabled || !textureLoaded ||
textureImage.isNull()) {
                float t = clamp01((Pworld.y + 1.2f) / 2.4f);
                Color3 low(0.78f, 0.78f, 0.82f);
                Color3 high(0.90f, 0.88f, 0.86f);
                return Color3(
                    low.r * (1.0f - t) + high.r * t,
                    low.g * (1.0f - t) + high.g * t,
                    low.b * (1.0f - t) + high.b * t
                );
            }
            Vec3 dir = normalize(PobjNorm);
            float ySafe = std::max(-1.0f, std::min(1.0f, dir.y));
            float u = 0.5f + std::atan2(dir.z, dir.x) / (2.0f *
3.14159265f);
            float v = 0.5f - std::asin(ySafe) / 3.14159265f;
            u *= textureRepeat;
            u = u - std::floor(u);
            v = clamp01(v);
            int tx = std::clamp(int(u * float(textureImage.width() -
1)), 0, textureImage.width() - 1);
            int ty = std::clamp(int(v * float(textureImage.height() -
1)), 0, textureImage.height() - 1);

            QColor qc = textureImage.pixelColor(tx, ty);
            return Color3(qc.red() / 255.0f, qc.green() / 255.0f,
qc.blue() / 255.0f);
        };

        auto shadePixel = [&](const Vec3& Pworld, const Vec3& Nworld,
const Color3& base) -> Color3 {
            Vec3 normal = normalize(Nworld);
            Vec3 V = normalize(cam.eye - Pworld);

            Color3 result(
                mat.ka.r * base.r,
```

```
                mat.ka.g * base.g,
                mat.ka.b * base.b
            );

            auto applyLight = [&](const LightData& light) -> Color3 {
                if (!light.enabled) return Color3(0, 0, 0);

                Vec3 L = normalize(light.pos - Pworld);
                float ndotl = std::max(0.0f, dot(normal, L));

                Color3 diffuse(
                    base.r * mat.kd.r * light.intensity.r * ndotl,
                    base.g * mat.kd.g * light.intensity.g * ndotl,
                    base.b * mat.kd.b * light.intensity.b * ndotl
                );

                Vec3 H = normalize(L + V);
                float ndoth = std::max(0.0f, dot(normal, H));
                float spec = std::pow(ndoth, mat.shininess);

                Color3 specular(
                    mat.ks.r * light.intensity.r * spec,
                    mat.ks.g * light.intensity.g * spec,
                    mat.ks.b * light.intensity.b * spec
                );

                return diffuse + specular;
            };

            result = result + applyLight(whiteLight);
            result = result + applyLight(blueLight);
            return clampColor(result);
        };

        // Ponemos un vertice del OBJ hasta la pantalla usando
modelo-vista-proyeccion
        auto projectVertex = [&](const Vec3& pObj, const Vec3& nObj) ->
ScreenV {
            Vec4 obj4(pObj.x, pObj.y, pObj.z, 1.0f);
            Vec4 objNorm4 = multiplyVec4(normalizeModel, obj4);
            Vec4 normal4(nObj.x, nObj.y, nObj.z, 0.0f);
            Vec4 world4 = multiplyVec4(model, obj4);
            Vec4 clip4 = multiplyVec4(mvp, obj4);
```

```
            Vec4 normalWorld4 = multiplyVec4(model, normal4);
            float safeW = (std::fabs(clip4.w) < 1e-6f)
                ? ((clip4.w >= 0.0f) ? 1e-6f : -1e-6f)
                : clip4.w;
            float invW = 1.0f / safeW;
            Vec4 ndc(clip4.x * invW, clip4.y * invW, clip4.z * invW,
1.0f);
            Vec4 screen4 = multiplyVec4(viewportMat, ndc);

            return {
                screen4.x,
                screen4.y,
                ndc.z,
                invW,
                xyz(objNorm4) * invW,
                xyz(world4) * invW,
                normalize(xyz(normalWorld4)) * invW
            };
        };

        // ========================
        // RASTERIZACIÓN TRIÁNGULO POR TRIÁNGULO
        // ========================

        for (const Face& f : faces) {
            // Leemos los tres vertices del triangulo en espacio objeto
            Vec3 p0Obj = vertices[f.a.v];
            Vec3 p1Obj = vertices[f.b.v];
            Vec3 p2Obj = vertices[f.c.v];

            // Tomamos las normales del OBJ o las de respaldo
calculadas
            Vec3 n0Obj = getNormalForCorner(f.a);
            Vec3 n1Obj = getNormalForCorner(f.b);
            Vec3 n2Obj = getNormalForCorner(f.c);

            //  Transoformamos los tres puntos a mundo para culling y
luz
            Vec3 p0World = xyz(multiplyVec4(model, Vec4(p0Obj.x,
p0Obj.y, p0Obj.z, 1.0f)));
            Vec3 p1World = xyz(multiplyVec4(model, Vec4(p1Obj.x,
p1Obj.y, p1Obj.z, 1.0f)));
```

```
            Vec3 p2World = xyz(multiplyVec4(model, Vec4(p2Obj.x,
p2Obj.y, p2Obj.z, 1.0f)));

            // Generamos la normal plana de la cara en el mundo
            Vec3 faceNormalWorld = normalize(cross(p1World - p0World,
p2World - p0World));

            // Hacemos la Backface culling para no rasterizar lo que no
mira a la camara
            Vec3 viewDir = normalize(cam.eye - p0World);
            if (dot(faceNormalWorld, viewDir) <= 0.0f) continue;

            // Hacemos la proyeccion completa de los tres vertices
            ScreenV s0 = projectVertex(p0Obj, n0Obj);
            ScreenV s1 = projectVertex(p1Obj, n1Obj);
            ScreenV s2 = projectVertex(p2Obj, n2Obj);

            float area = edge(s0.x, s0.y, s1.x, s1.y, s2.x, s2.y);
            if (std::fabs(area) < 1e-8f) continue;
            int minX = std::max(0, (int)std::floor(std::min({s0.x,
s1.x, s2.x})));
            int maxX = std::min(renderW - 1,
(int)std::ceil(std::max({s0.x, s1.x, s2.x})));
            int minY = std::max(0, (int)std::floor(std::min({s0.y,
s1.y, s2.y})));
            int maxY = std::min(renderH - 1,
(int)std::ceil(std::max({s0.y, s1.y, s2.y})));
            for (int y = minY; y <= maxY; ++y) {
                for (int x = minX; x <= maxX; ++x) {
                    float px = x + 0.5f;
                    float py = y + 0.5f;
                    float w0 = edge(s1.x, s1.y, s2.x, s2.y, px, py);
                    float w1 = edge(s2.x, s2.y, s0.x, s0.y, px, py);
                    float w2 = edge(s0.x, s0.y, s1.x, s1.y, px, py);
                    bool inside = (area > 0.0f)
                        ? (w0 >= 0.0f && w1 >= 0.0f && w2 >= 0.0f)
                        : (w0 <= 0.0f && w1 <= 0.0f && w2 <= 0.0f);
                    if (!inside) continue;
                    w0 /= area;
                    w1 /= area;
                    w2 /= area;
                    float interpInvW = s0.invW * w0 + s1.invW * w1 +
s2.invW * w2;
```

```
                    if (std::fabs(interpInvW) < 1e-8f) continue;
                    float depth = usePerspective
                        ? (s0.depth * s0.invW * w0 + s1.depth * s1.invW
* w1 + s2.depth * s2.invW * w2) / interpInvW
                        : (s0.depth * w0 + s1.depth * w1 + s2.depth *
w2);
                    int idx = y * renderW + x;
                    if (depth < zbuf[idx]) {
                        zbuf[idx] = depth;
                        Vec3 PobjNorm = (s0.objOverW * w0 + s1.objOverW
* w1 + s2.objOverW * w2) / interpInvW;
                        Vec3 Pworld = (s0.worldOverW * w0 +
s1.worldOverW * w1 + s2.worldOverW * w2) / interpInvW;
                        Vec3 Nworld = normalize((s0.normalOverW * w0 +
s1.normalOverW * w1 + s2.normalOverW * w2) / interpInvW);
                        Color3 base = sampleBaseColor(PobjNorm,
Pworld);
                        Color3 c = shadePixel(Pworld, Nworld, base);
                        img.setPixelColor(x, y, QColor(
                            int(clamp01(c.r) * 255.0f),
                            int(clamp01(c.g) * 255.0f),
                            int(clamp01(c.b) * 255.0f)
                        ));
                    }
                }
            }

            // Genera el dibujo del alambre si el usuario lo activa
            if (wireframe) {
                QPainter ip(&img);
                ip.setPen(QPen(QColor(240, 240, 240), 1));
                ip.drawLine(QPointF(s0.x, s0.y), QPointF(s1.x, s1.y));
                ip.drawLine(QPointF(s1.x, s1.y), QPointF(s2.x, s2.y));
                ip.drawLine(QPointF(s2.x, s2.y), QPointF(s0.x, s0.y));
            }
        }

        // Pintamos la imagen interna escalada a un tamaño real del
widget
        QPainter painter(this);
        painter.fillRect(rect(), QColor(18, 18, 26));
        painter.drawImage(rect(), img);
        painter.setPen(Qt::white);
```

```
        painter.drawText(10, 20, QString("OBJ con textura de imagen |
Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W"));
        painter.drawText(10, 40, QString("Vertices: %1 | Triangulos: %2
| Normales OBJ: %3 | Escala interna: %4%% | Textura imagen: %5 |
Repeticion U: %6x")
                                    .arg(vertices.size())
                                    .arg(faces.size())
                                    .arg(hasOBJNormals ? "Si" : "No,
usando normales calculadas")
                                    .arg(int(renderScale * 100.0f))
                                    .arg((textureEnabled &&
textureLoaded) ? "Activa" : (textureLoaded ? "Apagada" : "No cargada"))
                                    .arg(textureRepeat, 0, 'f', 1));
    }

private:
    int parseOBJIndex(const QString& s, int currentCount) const {
        int idx = s.toInt();
        if (idx > 0) return idx - 1;
        if (idx < 0) return currentCount + idx;
        return -1;
    }

    FaceVertex parseFaceVertexToken(const QString& token) const {
        FaceVertex fv;
        QStringList sub = token.split('/');

        if (sub.size() >= 1 && !sub[0].isEmpty()) {
            fv.v = parseOBJIndex(sub[0], (int)vertices.size());
        }
        if (sub.size() >= 3 && !sub[2].isEmpty()) {
            fv.vn = parseOBJIndex(sub[2], (int)objNormals.size());
        }
        return fv;
    }

    void computeFallbackNormals() {
        computedNormals.assign(vertices.size(), Vec3(0, 0, 0));

        for (const Face& f : faces) {
            if (f.a.v < 0 || f.b.v < 0 || f.c.v < 0) continue;

            Vec3 p0 = vertices[f.a.v];
```

```
            Vec3 p1 = vertices[f.b.v];
            Vec3 p2 = vertices[f.c.v];
            Vec3 fn = normalize(cross(p1 - p0, p2 - p0));

            computedNormals[f.a.v] = computedNormals[f.a.v] + fn;
            computedNormals[f.b.v] = computedNormals[f.b.v] + fn;
            computedNormals[f.c.v] = computedNormals[f.c.v] + fn;
        }

        for (Vec3& n : computedNormals) n = normalize(n);
    }

    Vec3 getNormalForCorner(const FaceVertex& fv) const {
        if (fv.vn >= 0 && fv.vn < (int)objNormals.size()) {
            return objNormals[fv.vn];
        }
        if (fv.v >= 0 && fv.v < (int)computedNormals.size()) {
            return computedNormals[fv.v];
        }
        return Vec3(0, 1, 0);
    }

    // Geometria del modelo
    std::vector<Vec3> vertices;
    std::vector<Vec3> objNormals;
    std::vector<Vec3> computedNormals;
    std::vector<Face> faces;

    // Camaras y materiales
    std::vector<CameraData> cameras;
    std::vector<Material> materials;

    // Luces
    LightData whiteLight;
    LightData blueLight;

    // Información del modelo para normalizarlo
    Vec3 minBound;
    Vec3 maxBound;
    Vec3 modelCenter;
    float normalizeScale = 1.0f;
    bool hasOBJNormals = false;
```

```
    // Estado actual de la escena
    float angleX = -0.25f;
    float angleY = 0.75f;
    bool usePerspective = true;
    bool wireframe = false;
    bool autoRotate = false;
    bool textureEnabled = true;
    bool textureLoaded = false;
    QImage textureImage;
    int materialIndex = 0;
    int cameraIndex = 0;
    float textureRepeat = 1.0f;
    float renderScale = 0.50f;
};

// ========================
// VENTANA PRINCIPAL CON CONTROLES QT
// ========================

class MainWindow : public QMainWindow {
public:
    MainWindow() {
        // Contenedor y layout principal
        auto* central = new QWidget;
        auto* mainLayout = new QHBoxLayout(central);

        // Área donde se dibuja el render
        render = new RenderWidget;

        // Panel lateral con controles.
        auto* panel = new QWidget;
        auto* panelLayout = new QVBoxLayout(panel);

        // Slider de la rotacion x
        auto* lblX = new QLabel("Rotacion X");
        auto* sliderX = new QSlider(Qt::Horizontal);
        sliderX->setRange(-180, 180);
        sliderX->setValue(-14);

        // Slider de la rotacion y
        auto* lblY = new QLabel("Rotacion Y");
        auto* sliderY = new QSlider(Qt::Horizontal);
        sliderY->setRange(-180, 180);
```

```
        sliderY->setValue(43);

        // Selector de proyeccion
        auto* projLabel = new QLabel("Proyeccion");
        auto* projectionBox = new QComboBox;
        projectionBox->addItem("Perspectiva");
        projectionBox->addItem("Ortogonal");

        // Selector del material
        auto* materialLabel = new QLabel("Material");
        auto* materialBox = new QComboBox;
        materialBox->addItem("Material A");
        materialBox->addItem("Material B");

        // Selector de la camara
        auto* cameraLabel = new QLabel("Camara");
        auto* cameraBox = new QComboBox;
        cameraBox->addItem("Camara 1");
        cameraBox->addItem("Camara 2");
        cameraBox->addItem("Camara 3");

        // Selector de la resolucion
        auto* scaleLabel = new QLabel("Resolucion interna");
        auto* scaleBox = new QComboBox;
        scaleBox->addItem("100%");
        scaleBox->addItem("75%");
        scaleBox->addItem("50%");
        scaleBox->setCurrentIndex(2);

        // Selector de cuantas veces se repite la textura de imagen.
        auto* textureInfoLabel = new QLabel("Textura de imagen");
        auto* textureNameLabel = new QLabel("Archivo:
textura_imagen.png");
        auto* repeatLabel = new QLabel("Repeticion horizontal");
        auto* repeatBox = new QComboBox;
        repeatBox->addItem("1x");
        repeatBox->addItem("2x");
        repeatBox->addItem("3x");

        // Casillas para seleccionar opciones variadas
        auto* wireframeCheck = new QCheckBox("Mostrar alambre");
        auto* autoRotateCheck = new QCheckBox("Rotacion automatica");
        auto* blueLightCheck = new QCheckBox("Luz azul activa");
```

```
        auto* textureCheck = new QCheckBox("Textura de imagen activa");
        blueLightCheck->setChecked(true);
        textureCheck->setChecked(true);

        // Botón para recargar el OBJ
        auto* reloadButton = new QPushButton("Recargar OBj y textura");

        // Agregamos controles al panel
        panelLayout->addWidget(lblX);
        panelLayout->addWidget(sliderX);
        panelLayout->addWidget(lblY);
        panelLayout->addWidget(sliderY);
        panelLayout->addWidget(projLabel);
        panelLayout->addWidget(projectionBox);
        panelLayout->addWidget(materialLabel);
        panelLayout->addWidget(materialBox);
        panelLayout->addWidget(cameraLabel);
        panelLayout->addWidget(cameraBox);
        panelLayout->addWidget(scaleLabel);
        panelLayout->addWidget(scaleBox);
        panelLayout->addWidget(textureInfoLabel);
        panelLayout->addWidget(textureNameLabel);
        panelLayout->addWidget(repeatLabel);
        panelLayout->addWidget(repeatBox);
        panelLayout->addWidget(wireframeCheck);
        panelLayout->addWidget(autoRotateCheck);
        panelLayout->addWidget(blueLightCheck);
        panelLayout->addWidget(textureCheck);
        panelLayout->addWidget(reloadButton);
        panelLayout->addStretch();

        // Ponemos el render y panel en la ventana principal
        mainLayout->addWidget(render, 1);
        mainLayout->addWidget(panel);

        setCentralWidget(central);
        setWindowTitle("Qt Proyecto OBJ");
        resize(1280, 780);

        // Conexiones de controles con el render
        QObject::connect(sliderX, &QSlider::valueChanged, render,
&RenderWidget::setAngleX);
```

```
        QObject::connect(sliderY, &QSlider::valueChanged, render,
&RenderWidget::setAngleY);
        QObject::connect(projectionBox,
&QComboBox::currentIndexChanged, this, [=](int idx) {
            render->setPerspective(idx == 0);
        });
        QObject::connect(materialBox, &QComboBox::currentIndexChanged,
render, [=](int idx) {
            render->setMaterialIndex(idx);
        });
        QObject::connect(cameraBox, &QComboBox::currentIndexChanged,
render, &RenderWidget::setCameraIndex);
        QObject::connect(scaleBox, &QComboBox::currentIndexChanged,
render, &RenderWidget::setRenderScaleIndex);
        QObject::connect(repeatBox, &QComboBox::currentIndexChanged,
render, &RenderWidget::setTextureRepeatIndex);
        QObject::connect(wireframeCheck, &QCheckBox::toggled, render,
&RenderWidget::setWireframe);
        QObject::connect(autoRotateCheck, &QCheckBox::toggled, render,
&RenderWidget::setAutoRotate);
        QObject::connect(blueLightCheck, &QCheckBox::toggled, render,
&RenderWidget::setBlueLightEnabled);
        QObject::connect(textureCheck, &QCheckBox::toggled, render,
&RenderWidget::setTextureEnabled);

        // Generamos un boton para volver a cargar el OBJ y su imagen
desde la carpeta del ejecutable
        QObject::connect(reloadButton, &QPushButton::clicked, this,
[=]() {
            QString base = QCoreApplication::applicationDirPath();
            QString objPath = base + "/bunny.obj";
            QString texPath = base + "/textura_cafe.png";
            if (!render->loadOBJ(objPath)) {
                QMessageBox::warning(this, "Error", "No se pudo
recargar bunny.obj");
            }
            if (!render->loadTextureImage(texPath)) {
                QMessageBox::warning(this, "Warning", "No se encontro
textura_cafe.png junto al ejecutable");
            }
        });

        // Timer para la animacion
```

```
        timer = new QTimer(this);
        QObject::connect(timer, &QTimer::timeout, render,
&RenderWidget::stepRotation);
        timer->start(16);

        // Cargamos la el OBJ y la imagen de textura.
        QString basePath = QCoreApplication::applicationDirPath();
        QString objPath = basePath + "/bunny.obj";
        QString texPath = basePath + "/textura_cafe.png";
        if (!render->loadOBJ(objPath)) {
            QMessageBox::warning(this, "Error", "No se encontro
bunny.obj junto al ejecutable.");
        }
        if (!render->loadTextureImage(texPath)) {
            QMessageBox::warning(this, "Warning", "No se encontro
textura_cafe.png junto al ejecutable");
        }
    }

private:
    RenderWidget* render = nullptr;
    QTimer* timer = nullptr;
};

// ========================
// FUNCIÓN PRINCIPAL
// ========================

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    MainWindow w;
    w.show();
    return app.exec();
}
```

Ejecutable

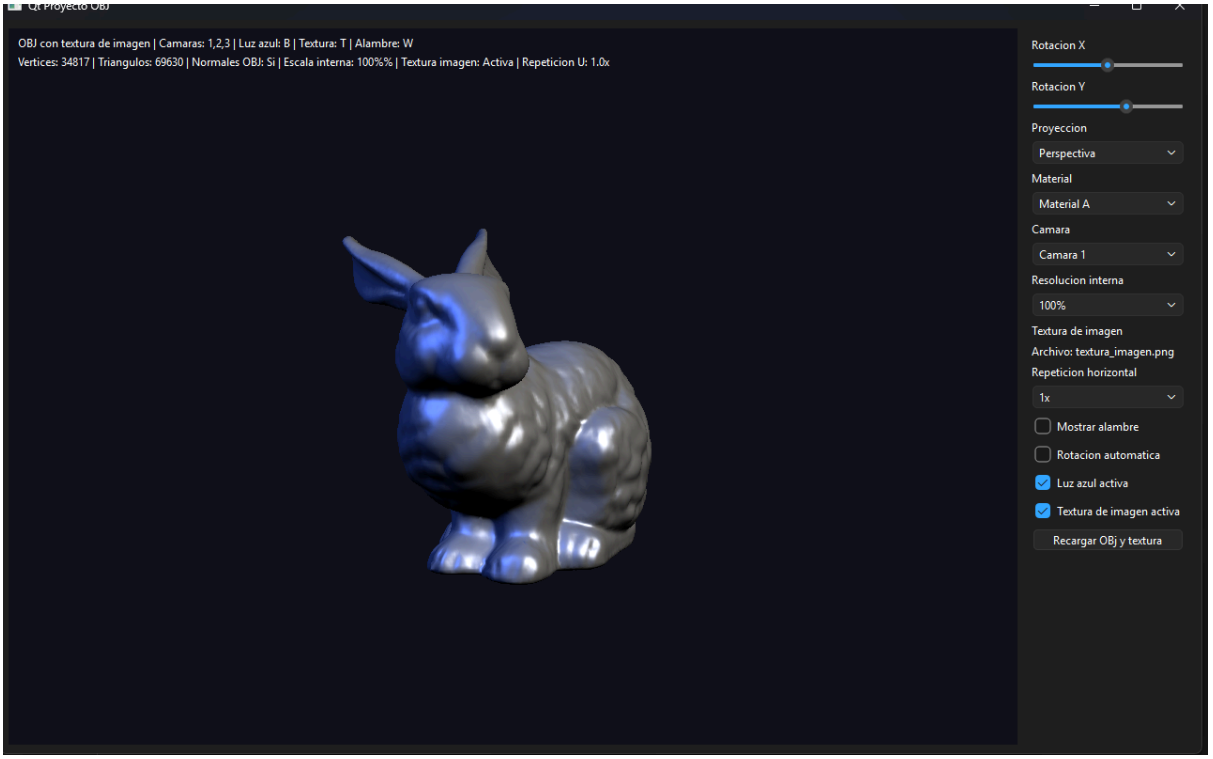
Para generar el ejecutable existen dos formas, una de ellas es desde el proyecto en Qt Creator abriendo el archivo CMakeLists.txt y compilarlo con la opción Build o la combinación Ctrl + B, una vez terminada la compilación Qt Creator genera el archivo ejecutable dentro de la carpeta de construcción del proyecto, la otra forma también puede generarse desde terminal usando los comandos cmake -S . -B build y cmake --build build --config Release, los cuales crean la carpeta de compilación y generan el ejecutable del programa, para que funcione correctamente, el ejecutable debe estar acompañado por el archivo OBJ y por la imagen de textura que se utilice en el renderizado.

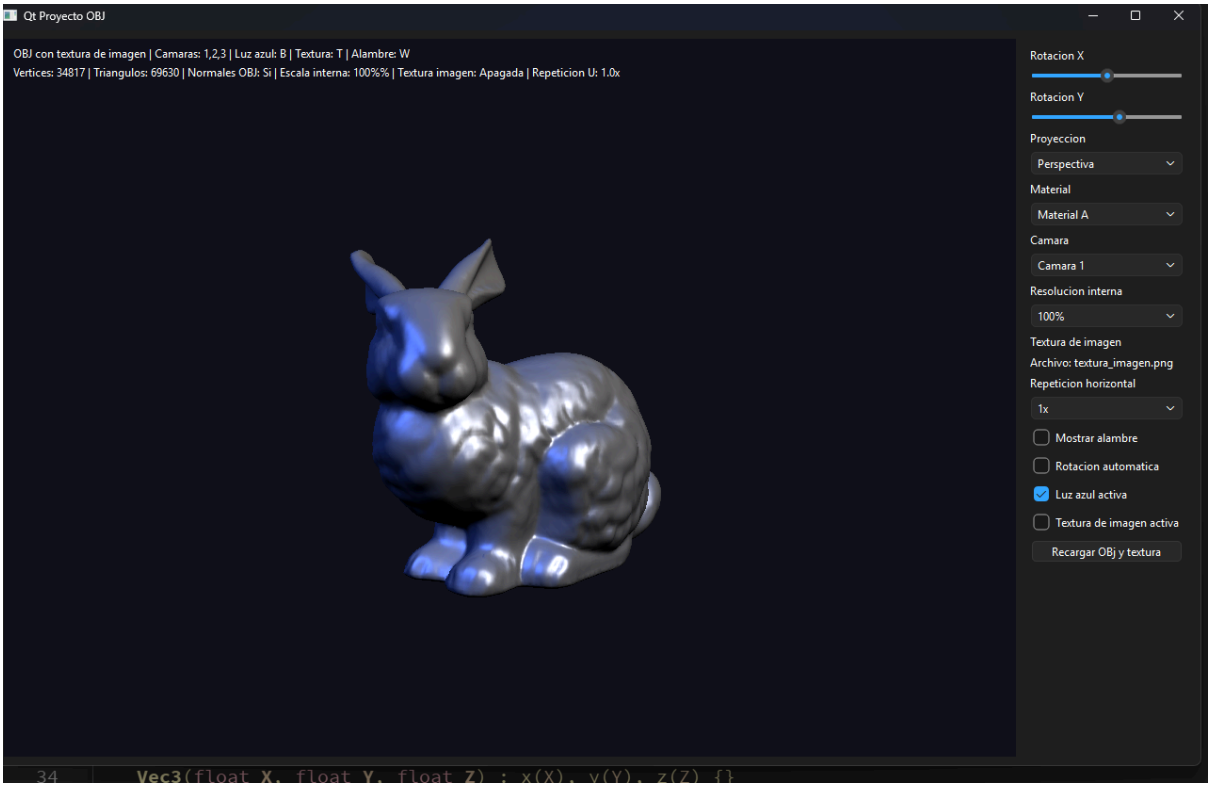
Uso del programa

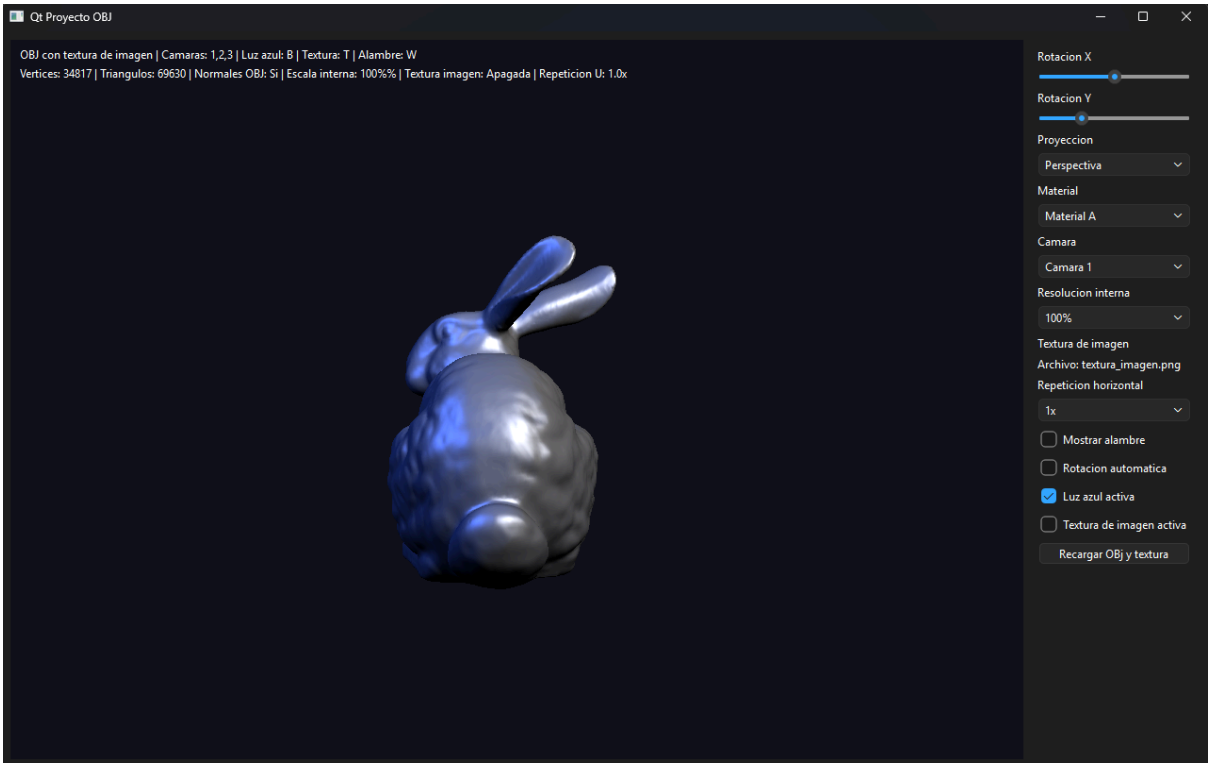
Tenemos que usar las teclas 1, 2, 3, B, W y T que junto a los controles del panel lateral se usan para las funciones que vienen implementadas en el programa. Las teclas 1, 2 y 3 permiten cambiar entre la cámara 1, la cámara 2 y la cámara 3 para observar el OBJ desde distintas posiciones, la tecla B activa o desactiva la luz azul para comparar cómo cambia la iluminación del modelo cuando se usa solo la luz blanca o cuando se combinan ambas luces, la tecla W activa o desactiva el modo alámbrico para mostrar la malla del OBJ y la tecla T activa o desactiva la textura aplicada sobre el modelo.

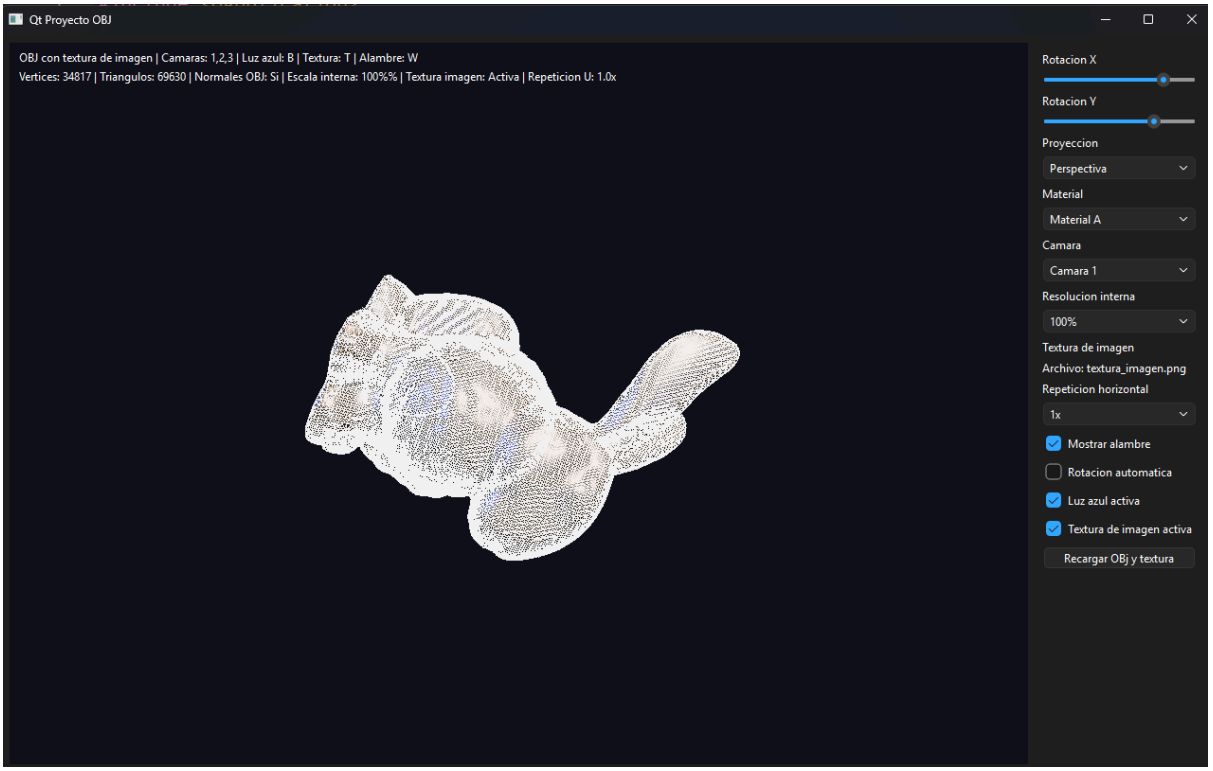
Además de estas teclas, en el panel lateral se pueden modificar la rotación en “x”, la rotación en “y”, el tipo de proyección entre perspectiva y ortogonal, el material activo, la cámara seleccionada, la resolución interna del render y también se puede recargar el archivo OBJ junto con la imagen de textura, cuando se inicia el programa se carga el archivo OBJ, se centra y escala automáticamente para que quede bien colocado en la escena, después se construye el pipeline de renderizado con matrices 4x4, se proyecta el modelo a la pantalla, se rasterizan sus triángulos, se calcula la iluminación con el modelo Phong-Blinn y finalmente se muestra la imagen resultante en la ventana. Si la textura está activada, el programa además calcula las coordenadas necesarias para leer el color correspondiente desde la imagen de textura y combinarlo con la iluminación final.

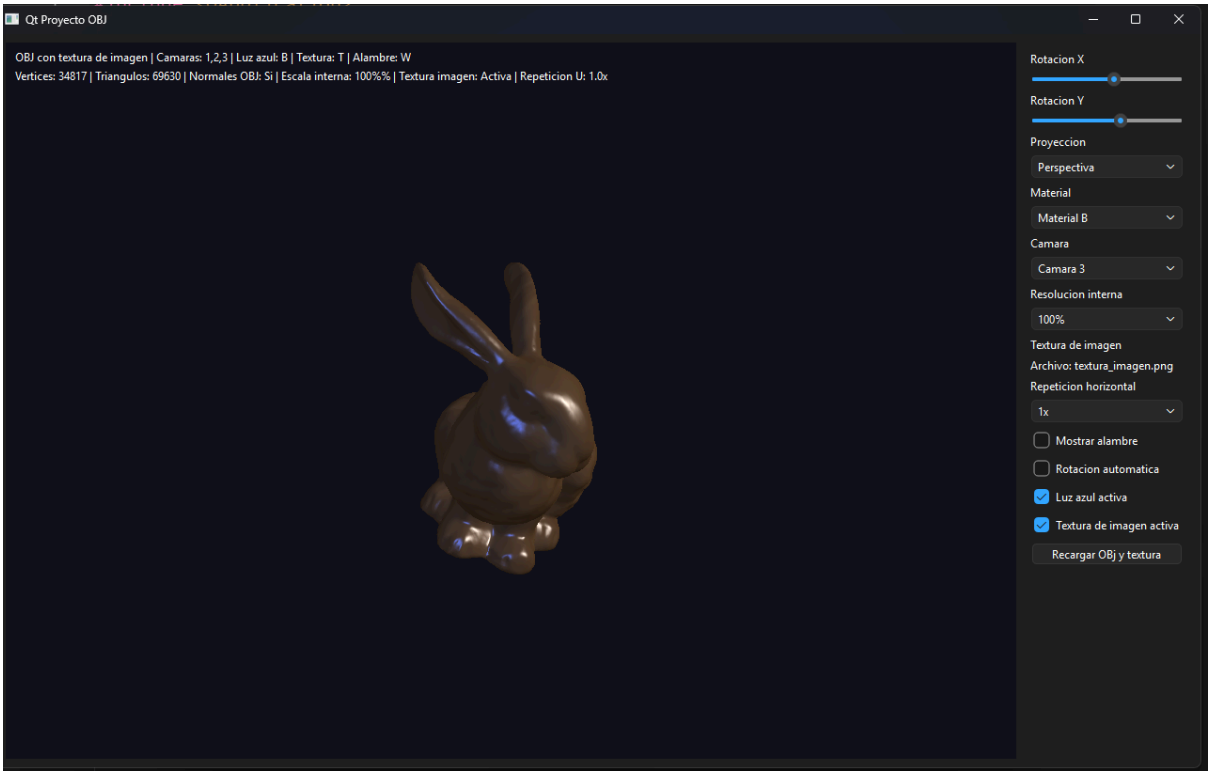
Imágenes

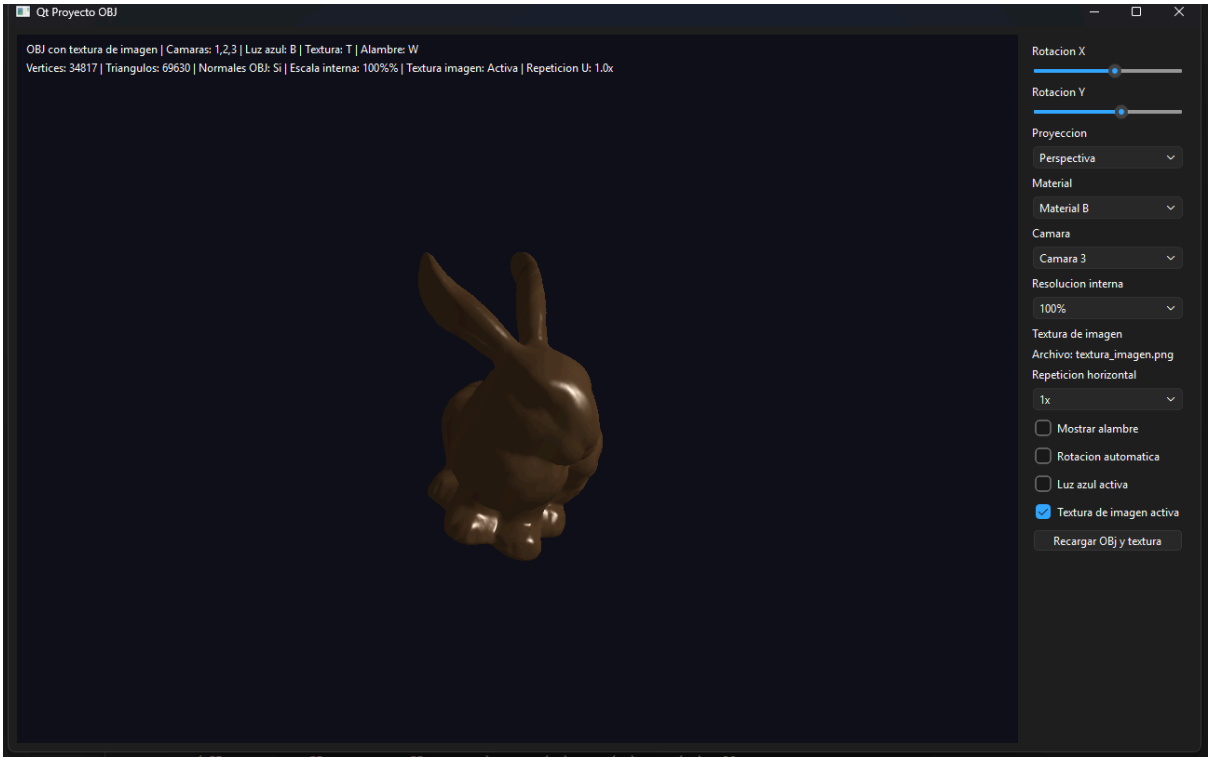

Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 100%% | Textura imagen: Activa | Repeticion U: 1.0x
Rotacion X
Rotacion Y
Proyeccion
Perspectiva
Material
Material A
Camara
Camara 1
Resolucion interna
100%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
1x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura

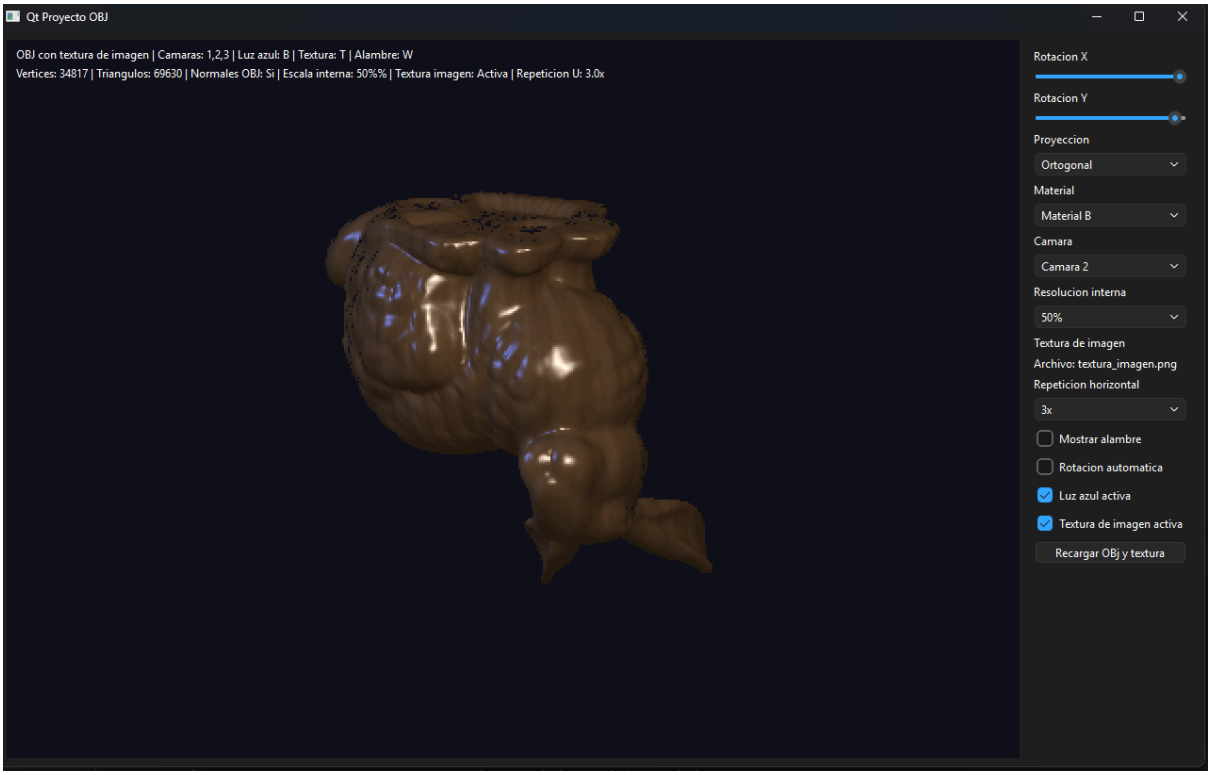

Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 100%% | Textura imagen: Apagada | Repeticion U: 1.0x
Rotacion X
Rotacion Y
Proyeccion
Perspectiva
Material
Material A
Camara
Camara 1
Resolucion interna
100%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
1x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura
34 Vec3(float X, float Y, float Z) : x(X), y(Y), z(Z) {}


Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 100%% | Textura imagen: Apagada | Repeticion U: 1.0x
Rotacion X
Rotacion Y
Proyeccion
Perspectiva
Material
Material A
Camara
Camara 1
Resolucion interna
100%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
1x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura


Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 100%% | Textura imagen: Activa | Repeticion U: 1.0x
Rotacion X
Rotacion Y
Proyeccion
Perspectiva
Material
Material A
Camara
Camara 1
Resolucion interna
100%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
1x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura


Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 100%% | Textura imagen: Activa | Repeticion U: 1.0x
Rotacion X
Rotacion Y
Proyeccion
Perspectiva
Material
Material B
Camara
Camara 3
Resolucion interna
100%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
1x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura


Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 100%% | Textura imagen: Activa | Repeticion U: 1.0x
Rotacion X
Rotacion Y
Proyeccion
Perspectiva
Material
Material B
Camara
Camara 3
Resolucion interna
100%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
1x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura


Qt Proyecto OBJ
OBJ con textura de imagen | Camaras: 1,2,3 | Luz azul: B | Textura: T | Alambre: W
Vertices: 34817 | Triangulos: 69630 | Normales OBJ: Si | Escala interna: 50%% | Textura imagen: Activa | Repeticion U: 3.0x
Rotacion X
Rotacion Y
Proyeccion
Ortogonal
Material
Material B
Camara
Camara 2
Resolucion interna
50%
Textura de imagen
Archivo: textura_imagen.png
Repeticion horizontal
3x
Mostrar alambre
Rotacion automatica
Luz azul activa
Textura de imagen activa
Recargar OBj y textura